



# History of Git: A Beginner's Guide

## *Introduction*

### *What is Git?*

- **Git** is a version control system that helps developers track changes in their code over time.
- It allows multiple people to work on a project simultaneously without conflicts.
- **Used by:** Software developers, open-source projects, and companies for collaboration.

### *Why Was Git Created?*

- Before Git, developers used **BitKeeper**, a proprietary version control system.
- In **2005**, BitKeeper's free license for Linux developers was revoked.
- **Linus Torvalds**, the creator of Linux, decided to build a **new, free, and powerful version control system**.
- Thus, **Git was born!**

### *Timeline: Evolution of Git*

#### **2005: The Birth of Git**

- Linus Torvalds designed **Git** for managing the Linux kernel's source code.
- Goals:
  - **Speed**
  - **Distributed development** (no central server required)
  - **Strong support for non-linear development** (branches & merging)



- **Security & integrity**

## **2006-2008: Early Adoption**

- **2006:** Git becomes **stable** and widely used in Linux development.
- **2007:** GitHub is launched, making Git adoption easier.
- **2008:** Major companies start using Git for software development.

## **2010s: Git Dominates the Industry**

- **2013:** Git becomes the most popular version control system.
- **2018:** Microsoft acquires GitHub for **\$7.5 billion**, proving Git's dominance.
- **2020s:** Git remains the **standard** for software development, used by **millions of developers worldwide**.

## ***How Git Works: Basic Concepts***

### ***1. Repository (Repo)***

- A **repository** is like a project folder where all code and history are stored.
- It tracks **every change** made to the files.

### ***2. Commit***

- A **commit** is a snapshot of the changes made to the code.
- Every commit has a unique **ID (hash)**.

### ***3. Branching & Merging***

- **Branching** allows developers to work on different features without affecting the main project.
- **Merging** combines different branches into one.



## ***Installing Git***

### ***Windows***

1. Download **Git for Windows** from [git-scm.com](https://git-scm.com).
2. Install and open **Git Bash**.
3. Verify installation:

```
git --version
```

### ***Linux (Ubuntu)***

```
sudo apt update  
sudo apt install git -y  
git --version
```

### ***MacOS***

```
brew install git  
git --version
```

## ***Basic Git Commands***

### ***Check Git Version***

```
git --version
```

### ***Set Up User Info***

```
git config --global user.name "Your Name"  
git config --global user.email "your_email@example.com"
```



## ***Initialize a Git Repository***

```
git init
```

## ***Clone a Repository***

```
git clone <repository_url>
```

## ***Add Changes to Staging Area***

```
git add .
```

## ***Commit Changes***

```
git commit -m "Your commit message"
```

## ***Push Changes to Remote Repository***

```
git push origin main
```

## ***Pull Latest Changes***

```
git pull origin main
```

## ***Know More: FAQs***

### ***1. Why is Git called a "distributed" version control system?***

- Unlike older systems, Git does not require a central server. Every developer has a full copy of the repository.

### ***2. How is Git different from GitHub?***

- **Git** is the version control tool.



- **GitHub** is a platform to host Git repositories online.

### ***3. What is the purpose of a commit?***

- A **commit** saves the current changes in the project with a unique ID, so you can track progress or roll back if needed.

### ***4. How do I check the history of commits?***

```
git log
```

### ***5. How can I undo a commit?***

```
git revert <commit_id>
```

## ***Conclusion***

Git revolutionized **software development** by providing a fast, reliable, and distributed version control system. Understanding Git will help you collaborate efficiently and manage projects like a pro!